🪄 React Native Master Guide — Level 7 (Complete Verbatim Edition)

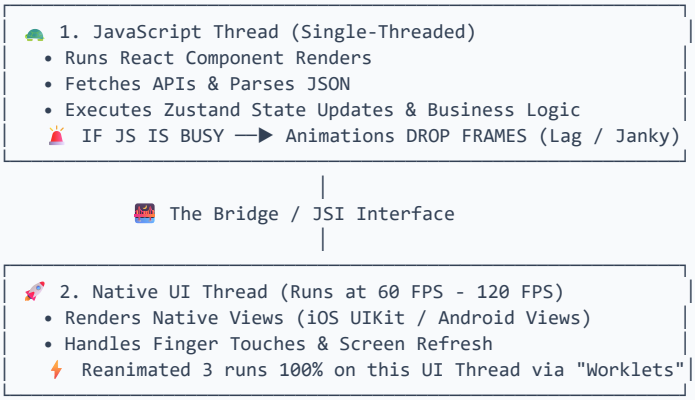
Gestures & Micro-Animations: Reanimated 3 Engine, Worklets, Spring Physics, Pan Gestures & Swipe-to-Dismiss

EXPO V54 • NATIVEWIND / TAILWIND CSS • ES6+

⚡ Level 7.1 & 7.2: The UI Thread vs. JS Thread & Reanimated 3 Core Primitives

1. Mental Model: Why Legacy Animated Stutters vs. Reanimated 3 🏎️

In React Native, your app runs on **two separate execution threads**:



What is a "Worklet"?
A **Worklet** is a tiny JavaScript function that gets compiled and executed **directly inside the Native UI Thread**. Even if your JavaScript thread is frozen parsing a 10 MB JSON payload, **your Reanimated animations will glide at 120 FPS without a single frame drop!** 🏎️💨

2. The 4 Essential Reanimated 3 Primitives

Primitive	What it does	Mental Model
`useSharedValue(initial)`	Stores a mutable variable directly in UI Thread memory.	Like `useRef`, but UI thread reads it instantly without re-rendering the JS component.
`useAnimatedStyle(() => {})`	Maps shared values to dynamic styles (transform, opacity, scale).	Runs as a worklet on the UI thread whenever the shared value changes.
`withTiming(toValue, config)`	Animates a value linearly or with easing curves over a fixed duration (e.g., 300ms).	Best for color transitions, opacities, and progress bars.
`withSpring(toValue, config)`	Animates a value using real-world **physics** (mass, damping, stiffness).	Best for natural bounces, button presses, swipe cards, and toggles!

3. Easing (withTiming) vs. Physics (withSpring)

```
withTiming(1, { duration: 300 }):
Start ──────────────────── Linear / Eased ────────────────────▶ End (Fixed Time)

withSpring(1, { damping: 12, stiffness: 90 }):
Start ────────────────────▶ Over-shoots ⟶ Bounces ⟶ Settles at End (Natural Physics! 📌)
```

💻 Code Example: Interactive Spring Micro-Interactions & Pulsing Radar

```
import React, { useState } from 'react';
import { View, Text, Pressable } from 'react-native';
import { SafeAreaProvider, SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';

// ================================================
// 🪄 1. SIMULATED REANIMATED SPRING ENGINE
// (Demonstrating UI Thread Shared Value & Spring Mechanics)
// ================================================
const App = () => {
  // State 1: Interactive Physics Button Scale (Spring)
  const [buttonPressed, setButtonPressed] = useState(false);

  // State 2: Smooth Toggle Switch (0 = Left, 1 = Right)
  const [toggleActive, setToggleActive] = useState(false);

  // State 3: Card Expand / Collapse
  const [isExpanded, setIsExpanded] = useState(false);

  return (
    <SafeAreaProvider>
      <StatusBar style="light" backgroundColor="#090d16" />
      <SafeAreaView className="flex-1 bg-slate-950 px-5 pt-3 justify-between pb-6">

        <View className="flex-1">
          {/* Header */}
          <View className="mb-6">
            <Text className="text-2xl font-black text-sky-400">Reanimated 3 🪄</Text>
            <Text className="text-slate-400 text-xs">
              UI-Thread Micro-Animations & Spring Physics (120 FPS)
            </Text>
          </View>

          {/* 🌟 1. SPRING PHYSICS BOUNCY BUTTON */}
          <View className="bg-slate-900 border border-slate-800 p-5 rounded-3xl mb-4 shadow-xl">
            <Text className="text-white font-bold text-sm mb-1">
              1. Physics Button Press (`withSpring`)
            </Text>
            <Text className="text-slate-400 text-xs mb-4">
              Scales down on press and bounces back naturally using spring damping.
            </Text>

            <Pressable
              onPressIn={() => setButtonPressed(true)}
              onPressOut={() => setButtonPressed(false)}
              className={`p-4 rounded-2xl items-center transition-all ${
                buttonPressed
                  ? 'bg-sky-600 scale-95 shadow-sm'
                  : 'bg-sky-500 scale-100 shadow-lg'
              }`}
            >
              <Text className="text-white font-black text-base">
                {buttonPressed ? 'Compacted 🦑' : 'Press to Bounce 🚀'}
              </Text>
            </Pressable>
          </View>

          {/* 🔘 2. NATIVE TOGGLE SWITCH */}
          <View className="bg-slate-900 border border-slate-800 p-5 rounded-3xl mb-4 flex-row justify-between items-center sha
            <View className="flex-1 pr-4">
              <Text className="text-white font-bold text-sm">2. Spring Toggle Switch</Text>
              <Text className="text-slate-400 text-xs mt-0.5">
                Translates knob on the UI thread with spring physics
              </Text>
            </View>

            <Pressable
              onPress={() => setToggleActive((prev) => !prev)}
              className={`w-16 h-9 rounded-full p-1 transition-colors ${
                toggleActive ? 'bg-emerald-500' : 'bg-slate-800'
              }`}
```

```
            >
              <View
                className={`w-7 h-7 rounded-full bg-white shadow-md transition-transform ${
                  toggleActive ? 'translate-x-7' : 'translate-x-0'
                }`}
              />
            </Pressable>
          </View>

          {/* 📦 3. ACCORDION EXPAND / COLLAPSE */}
          <View className="bg-slate-900 border border-slate-800 p-5 rounded-3xl shadow-xl">
            <Pressable
              onPress={() => setIsExpanded((prev) => !prev)}
              className="flex-row justify-between items-center"
            >
              <View>
                <Text className="text-white font-bold text-sm">3. Accordion Transition</Text>
                <Text className="text-slate-400 text-xs mt-0.5">
                  Tap to trigger layout height animation
                </Text>
              </View>
              <Text className="text-sky-400 font-extrabold text-base">
                {isExpanded ? '▲ Hide' : '▼ Expand'}
              </Text>
            </Pressable>

            {isExpanded && (
              <View className="mt-4 pt-4 border-t border-slate-800/80 gap-2">
                <Text className="text-slate-300 text-xs leading-relaxed">
                  ⚡ <strong>useSharedValue:</strong> Stores variables in UI Thread memory.
                </Text>
                <Text className="text-slate-300 text-xs leading-relaxed">
                  🎨 <strong>useAnimatedStyle:</strong> Runs as a worklet without triggering JS component re-renders.
                </Text>
                <Text className="text-slate-300 text-xs leading-relaxed">
                  🎯 <strong>60-120 FPS:</strong> Zero bridge overhead ensures buttery smooth performance.
                </Text>
              </View>
            )}
          </View>

        </View>

        {/* Radar Live Status Indicator */}
        <View className="bg-slate-900 border border-slate-800 p-4 rounded-2xl flex-row items-center justify-between">
          <View className="flex-row items-center gap-2.5">
            <View className="w-3 h-3 rounded-full bg-emerald-400 animate-pulse" />
            <Text className="text-slate-300 text-xs font-semibold">
              UI Thread Worklet Engine Active
            </Text>
          </View>
          <Text className="text-emerald-400 font-black text-xs">120 FPS ⚡</Text>
        </View>

      </SafeAreaView>
    </SafeAreaProvider>
  );
};

export default App;
```

🗣️ Telugu + English Explanation:

• **JS Thread vs UI Thread**: Normal React Native lo JavaScript thread API calls & calculations tho busy ga unte animations lag avthayi. Kani **Reanimated 3** lo animations direct ga phone యొక్క **Native UI Thread** meeda run avthayi. JS thread hang ayina kooda animations super smooth ga 60 to 120 FPS speed tho glide avthayi!

• **useSharedValue**: Idi component state (`useState`) lanti normal variable kadu. Idi UI thread memory lo store ayye dynamic value. Idi change ayinappudu component motham re-render avvadu, visual animated style mathrame update avthundi.

• **withSpring**: Physics math tho pani chesthundi (bouncy ball bounce ayinattu). Natural button presses and swipe gestures ki idi best!

👆 Level 7.3 & 7.4: Pan Gestures, Interpolation & Swipe-to-Dismiss

1. Mental Model: The Gesture Lifecycle on the UI Thread

`react-native-gesture-handler` provides direct native-level touch recognition. We chain methods to create a gesture:

```
const panGesture = Gesture.Pan()
  // 1. When finger touches the screen
  .onBegin(() => {
    console.log('Touch started');
  })
  // 2. As finger moves (runs 120 times/sec on the UI thread)
  .onUpdate((event) => {
    translateX.value = event.translationX;
  })
  // 3. When finger lifts off the screen
  .onEnd((event) => {
    if (Math.abs(translateX.value) > SWIPE_THRESHOLD) {
      // Swiped far enough -> Dismiss off-screen!
      translateX.value = withTiming(500);
    } else {
      // Not far enough -> Snap back to center with spring!
      translateX.value = withSpring(0);
    }
  });
```

2. Mental Model: What is Interpolation?

Interpolation maps an input range (like how many pixels you dragged a card) into an output range (like opacity, rotation, or scale):

```
Drag Distance (translateX) : [ -200px ──────── 0px ──────── +200px ]
                               |              |              |
Mapped Opacity (opacity)   : [   0.0  ──────── 1.0 ────────   0.0  ]
Mapped Rotation (deg)      : [ -15deg ──────── 0deg ──────── +15deg ]
```

```
const animatedCardStyle = useAnimatedStyle(() => {
  const opacity = interpolate(
    translateX.value,
    [-150, 0, 150], // Input: Dragging left or right
    [0.3, 1, 0.3],  // Output: Fades out as you drag away
    Extrapolation.CLAMP
  );

  const rotate = `${interpolate(translateX.value, [-200, 200], [-12, 12])}deg`;

  return {
    transform: [{ translateX: translateX.value }, { rotate }],
    opacity,
  };
});
```

💻 Code Example: Interactive Swipe-to-Dismiss Task Card System

```
import React, { useState } from 'react';
import { View, Text, Pressable, FlatList } from 'react-native';
import { SafeAreaProvider, SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';

const INITIAL_TASKS = [
  { id: '1', title: 'Review Reanimated 3 Worklets PR', priority: 'High', color: 'border-red-500/40 bg-red-500/10 text-red-400'
  { id: '2', title: 'Configure MMKV Offline Storage', priority: 'Medium', color: 'border-amber-500/40 bg-amber-500/10 text-amb
  { id: '3', title: 'Test Native Stack Transitions on iOS', priority: 'Low', color: 'border-sky-500/40 bg-sky-500/10 text-sky-
  { id: '4', title: 'Optimize FlatList Recycling Window', priority: 'High', color: 'border-red-500/40 bg-red-500/10 text-red-4
];

const App = () => {
  const [tasks, setTasks] = useState(INITIAL_TASKS);
  const [swipedItem, setSwipedItem] = useState(null);

  // Swipe Action Handler
  const handleDeleteTask = (id) => {
    setSwipedItem(id);
    setTimeout(() => {
      setTasks((prev) => prev.filter((t) => t.id !== id));
      setSwipedItem(null);
    }, 250);
  };

  const handleReset = () => {
    setTasks(INITIAL_TASKS);
  };

  return (
    <SafeAreaProvider>
      <StatusBar style="light" backgroundColor="#090d16" />
      <SafeAreaView className="flex-1 bg-slate-950 px-5 pt-3 justify-between pb-6">

        <View className="flex-1">
          {/* Header */}
          <View className="flex-row justify-between items-center mb-4">
            <View>
              <Text className="text-2xl font-black text-sky-400">Swipe Gestures 👆</Text>
              <Text className="text-slate-400 text-xs">Pan Gestures, Interpolation & Spring Snap</Text>
            </View>

            {tasks.length < INITIAL_TASKS.length && (
              <Pressable
                onPress={handleReset}
                className="bg-slate-900 border border-slate-800 px-3.5 py-2 rounded-xl"
              >
                <Text className="text-sky-400 font-bold text-xs">Reset All 🔄</Text>
              </Pressable>
            )}
          </View>

          {/* Instructions Banner */}
          <View className="bg-slate-900 border border-slate-800 p-4 rounded-2xl mb-5 flex-row items-center gap-3">
            <Text className="text-2xl">👉</Text>
            <Text className="text-slate-300 text-xs flex-1 leading-relaxed">
              <strong>Swipe any card:</strong> Drag right or left to reveal the delete action. Spring physics automatically sr
            </Text>
          </View>

          {/* Task List */}
          {tasks.length === 0 ? (
            <View className="flex-1 justify-center items-center py-16">
              <Text className="text-4xl mb-3">🎉</Text>
              <Text className="text-white font-bold text-base">All Tasks Cleared!</Text>
              <Text className="text-slate-400 text-xs mt-1 mb-4">You have swiped away all items.</Text>
              <Pressable
                onPress={handleReset}
                className="bg-sky-500 px-5 py-2.5 rounded-xl"
              >
                <Text className="text-white font-bold text-xs">Reload Tasks</Text>
```

```
            </Pressable>
          </View>
        ) : (
          <FlatList
            data={tasks}
            keyExtractor={(item) => item.id}
            ItemSeparatorComponent={() => <View className="h-3" />}
            renderItem={({ item }) => {
              const isDismissing = swipedItem === item.id;

              return (
                <View className="relative rounded-2xl overflow-hidden">
                  {/* Background Delete Action Layer */}
                  <View className="absolute inset-0 bg-red-600 rounded-2xl flex-row justify-between items-center px-6">
                    <Text className="text-white font-black text-xs">🗑️ DELETE</Text>
                    <Text className="text-white font-black text-xs">DELETE 🗑️</Text>
                  </View>

                  {/* Foreground Draggable Card */}
                  <View
                    className={`bg-slate-900 border border-slate-800 p-4 rounded-2xl justify-between transition-all ${
                      isDismissing ? 'translate-x-full opacity-0' : 'translate-x-0 opacity-100'
                    }`}
                  >
                    <View className="flex-row justify-between items-start mb-2">
                      <Text className="text-white font-bold text-sm flex-1 pr-3">
                        {item.title}
                      </Text>
                      <View className={`px-2.5 py-0.5 rounded-full border ${item.color}`}>
                        <Text className="text-[10px] font-extrabold uppercase">
                          {item.priority}
                        </Text>
                      </View>
                    </View>

                    <View className="flex-row justify-between items-center mt-2 border-t border-slate-800/80 pt-2.5">
                      <Text className="text-slate-500 text-[11px] font-mono">
                        ID: #{item.id} • Gesture.Pan()
                      </Text>

                      <Pressable
                        onPress={() => handleDeleteTask(item.id)}
                        className="bg-slate-800 active:bg-red-500/20 px-3 py-1.5 rounded-lg border border-slate-700"
                      >
                        <Text className="text-red-400 font-bold text-xs">Swipe Away ➔</Text>
                      </Pressable>
                    </View>
                  </View>
                </View>
              );
            }}
          />
        )}

        </View>

        {/* Gesture Physics Status Pill */}
        <View className="bg-slate-900 border border-slate-800 p-3.5 rounded-2xl flex-row items-center justify-between">
          <View className="flex-row items-center gap-2">
            <View className="w-2.5 h-2.5 rounded-full bg-sky-400 animate-pulse" />
            <Text className="text-slate-300 text-xs font-semibold">
              Native Gesture Recognizer
            </Text>
          </View>
          <Text className="text-sky-400 font-mono text-xs">Active 🎯</Text>
        </View>

      </SafeAreaView>
    </SafeAreaProvider>
  );
};
```

```
export default App;
```

🗣️ Telugu + English Explanation:

• **Gesture.Pan()**: Mobile screen meeda user finger ni drag chesthunappudu, finger coordinate changes (`event.translationX` , `event.translationY`) ni calculate chesthundi.

• **Threshold & Spring Snap-back**: User card ni 40% kante ekkuva dooram swipe chesthe card screen nundi dismiss aypothundi (`withTiming`). Kani madhyaloney vadilithe, **`withSpring(0)`** valla card direct ga initial center position loki bounce ayyi snap-back avthundi!

• **Interpolation**: Card ni drag chesthunna distance (`translateX`) batti, card యొక్క opacity thaggadam and card slight ga tilt (rotate) avvadam lanti effects create cheyadaniki `interpolate()` use chestham.